



Recursion — Master Revision Sheet

Step 7 · P01–P24 · Striver A2Z DSA Course · 24 Problems

⚡ 60-SECOND INDEX

#	PROBLEM	LC	PATTERN	TIME / SPACE
P01	Recursive atoi	8	Parameterized tail recursion	$O(N)$ / $O(N)$
P02	Pow(x, n)	50	Binary exponentiation	$O(\log N)$ / $O(\log N)$
P03	Count Good Numbers	1922	Math + modular binary exp	$O(\log N)$ / $O(1)$
P04	Sort Stack	GFG	Two-function recursive (peel + insert)	$O(N^2)$ / $O(N)$
P05	Reverse Stack	GFG	Two-function recursive (peel + insertAtBottom)	$O(N^2)$ / $O(N)$
P06	Binary Strings No 11	GFG	Constrained string generation	$O(\text{Fib}(n) \times n)$ / $O(n)$
P07	Generate Parentheses	22	Open/close counter guards	$O(4^n / n)$ / $O(n)$
P08	Power Set	78	Include/exclude OR bitmask	$O(2^N \cdot N)$ / $O(N)$
P09	Count Subsequences Sum K	GFG	Include/exclude → count (+)	$O(2^N)$ / $O(N)$
P10	Check Subsequence Sum K	GFG	Include/exclude → exists ()	$O(2^N)$ / $O(N)$
P11	Combination Sum I	39	Backtrack loop, stay at idx (reuse)	$O(k \times 2^t)$ / $O(t)$
P12	Combination Sum II	40	Backtrack loop, i+1, dup skip	$O(2^N)$ / $O(N)$
P13	Subset Sums I	GFG	Include/exclude, carry sum as int	$O(2^N \cdot N)$ / $O(N)$
P14	Subsets II	90	Push-on-entry + dup skip	$O(2^N \cdot N)$ / $O(N)$
P15	Combination Sum III	216	Fixed pool 1–9, double base case	$O(C(9, k) \cdot k)$ / $O(k)$
P16	Letter Combinations	17	Digit → letters map, recurse idx+1	$O(4^N \cdot N)$ / $O(N)$
P17	Palindrome Partitioning	131	Partition loop + isPalin check	$O(2^N \cdot N)$ / $O(N)$
P18	Word Search	79	DFS + in-place '#' mark + restore	$O(M \cdot N \cdot 4^L)$ / $O(L)$
P19	N-Queens	51	Column-by-column + 3 hash arrays	$O(N!)$ / $O(N)$
P20	Rat in a Maze	GFG	DFS + visited[] + D/L/R/U order	$O(4^N \cdot N^2)$ / $O(N^2)$
P21	Word Break	139	Partition loop + dict lookup + memo	$O(N^3)$ / $O(N)$
P22	M Coloring Problem	GFG	Node-by-node, try 1 → M colors	$O(M^N \cdot N)$ / $O(N)$
P23	Sudoku Solver	37	Find '.' + try '1' → '9' + row/col/box	$O(9^{\text{empty}})$ / $O(1)$
P24	Expression Add Operators	282	DFS on splits + mul trick (last_operand)	$O(N \cdot 4^N)$ / $O(N)$



PATTERN TOOLBOX

Include / Exclude (P08–P10, P13)

Two branches per index: take or skip. Combine with + (count), || (check), or collect (print). `sum==0` base before `ind==n` for positive-array pruning.

Backtracking Loop (P11–P17)

```
for i=idx; push arr[i]; recurse(i+1); pop. Use i (not i+1) for unlimited reuse (P11). Dup skip: i>idx && arr[i]==arr[i-1].
```

Push-on-Entry vs Push-at-Base

In-Place Mark + Restore (P18, P20)

Entry = all subsets including empty (P14). Base = only complete results (P11, P12, P17). Know which is required before coding.

Mark '#' or visited=1 BEFORE recursing all directions. Restore after — the restore IS the backtrack.

Binary Exponentiation (P02, P03)

`half=f(n/2); half*half [x if odd]`. Store in `half` and reuse — calling twice makes it $O(N)$. Use `long` for `INT_MIN` negation.

Two-Function Stack (P04, P05)

Main function peels elements. Helper (insert / insertAtBottom) tunnels through stack and restores on way up.

Hash Arrays for $O(1)$ Conflict (P19)

`leftRow[r]`, `lowerDiag[r+c]`, `upperDiag[n-1+c-r]`. Diagonal key: `row+col` invariant along $\swarrow$; `col-row` along $\searrow$. Array size: $2n-1$.

Last-Operand Tracking (P24)

Mul trick: `val = val - last + last*cur`. After `-c`: `last = -c` for correct chained multiply. Carry `last_operand` at every step.

PROBLEM-BY-PROBLEM

P01 — Recursive atoi $O(N)/O(N)$

Problem: Convert string to integer recursively. Skip spaces, read sign, recurse digit-by-digit. "`-42abc`" $\rightarrow$ `-42`.

Recall: Replace `while(isdigit)` loop with `solve(i+1, sign, num*10+d)`. Overflow check BEFORE multiply: `num > (INT_MAX - d) / 10`.

Key: `myAtoi` handles spaces + sign iteratively; `solve` handles digits recursively. Pass `string&`, use `long long num`.

Trap: Calling `back()` on empty string is UB; pass `string&` to avoid copying per frame.

P02 — Pow(x, n) $O(\log N)/O(\log N)$

Problem: Implement `pow(x,n)`. `x=2.0, n=10  $\rightarrow$  1024.0`. `n=-2  $\rightarrow$  0.25`.

Recall: `half = helper(x, n/2)`. Even: `half*half`. Odd: `half*half*x`. Negative: `x=1/x, N=-N` using `long`.

Trap: `long N=n` mandatory — `-INT_MIN` overflows `int`. Computing `helper(x,n/2)*helper(x,n/2)` (calling twice) collapses to $O(N)$.

P03 — Count Good Numbers $O(\log N)/O(1)$

Problem: Count length-`n` digit strings where even-indexed = even digit, odd-indexed = prime digit. `n=4  $\rightarrow$  400`.

Recall: Even pos $\rightarrow$ 5 choices; odd $\rightarrow$ 4 choices. Answer = $5^{\lceil n/2 \rceil} \times 4^{\lfloor n/2 \rfloor} \% \text{MOD}$. Binary exp: `n&1, x=(x*x)%MOD, n>=1`.

Trap: Apply `%MOD` after every multiplication inside the loop. Final product also needs `%MOD`.

P04 — Sort Stack using Recursion $O(N^2)/O(N)$

Problem: Sort a stack (largest on top) using only recursion. `[4,1,3,2]  $\rightarrow$  [1,2,3,4]` (top=4).

Recall: `sortStack = peel top  $\rightarrow$  sort rest  $\rightarrow$  insert(top)`. `insert`: if `empty || top ≤ temp` push; else pop top, recurse, push back.

Trap: Condition is `≤` not `<` — equal elements would loop forever with `<`. Check `s.empty()` before `s.top()`.

P05 — Reverse Stack using Recursion $O(N^2)/O(N)$

Problem: Reverse a stack using only recursion. `[4,1,3,2]` (top=2) $\rightarrow$ top=4.

Recall: `reverseStack = peel top  $\rightarrow$  reverse rest  $\rightarrow$  insertAtBottom(top)`. `insertAtBottom`: pop all $\rightarrow$ push val $\rightarrow$ restore all.

Trap: Mirror of P04. Forgetting `push(topVal)` in `insertAtBottom` after recursion drops elements silently.

P06 — Binary Strings without Consecutive 1s $O(\text{Fib}(n) \cdot n)/O(n)$

Problem: Generate all binary strings of length `n` with no two consecutive 1s. `n=3  $\rightarrow$  000 001 010 100 101`.

Recall: Always branch `'0'`. Branch `'1'` only if `curr.empty() || curr.back() != '1'`. Base: `len==n`.

Trap: Check `curr.empty()` BEFORE `curr.back()` — `back()` on empty string is UB. Count = `Fibonacci(n+2)`.

P07 — Generate Parentheses $O(4^n/n)/O(n)$

Problem: Generate all valid combinations of n pairs of parentheses. $n=2 \rightarrow ["()()", "()(())"]$.

Recall: Add '(' if $open < n$. Add ')' if $close < open$. Base: $len == 2n \rightarrow push$.

Trap: $close < open$ not $\leq$ — at balance point must open, not close. Count of valid strings = Catalan(n).

P08 — Power Set / All Subsequences $O(2^N \cdot N)/O(N)$

Problem: Generate all subsequences (including empty) of a string. "abc" $\rightarrow$ 8 strings.

Recall: **Recursion:** exclude (recurse), then include (push, recurse, pop_back). **Bitmask:** loop mask $0 \rightarrow 2^n - 1$; include $s[i]$ if bit i set.

Trap: Pass curr by reference + pop_back for $O(N)$ space. By value copies entire string each call.

P09 — Count Subsequences with Sum K $O(2^N)/O(N)$

Problem: Count subsequences summing to K (positive integers). $[1, 2, 3, 4, 5]$, $K=5 \rightarrow 3$.

Recall: $sum == 0 \rightarrow 1$ FIRST. $sum < 0 \mid \mid ind == n \rightarrow 0$. Return $func(ind+1, sum - arr[ind]) + func(ind+1, sum)$.

Trap: $sum == 0$ before $ind == n$ = early exit for positive arrays. Remove $sum < 0$ prune for arrays with negative elements.

P10 — Check if Subsequence Sum K Exists $O(2^N)/O(N)$

Problem: Does any subsequence sum to K ? $[1, 2, 3, 4]$, $K=5 \rightarrow true$.

Recall: Base $ind == n \rightarrow sum == 0$. Return $func(...) \mid \mid func(...)$ — short-circuit stops on first found.

Trap: $|$ (bitwise OR) evaluates both branches always. Use $\mid \mid$ for existence checks.

P11 — Combination Sum I $O(k \cdot 2^t)/O(t)$

Problem: Distinct candidates, unlimited reuse, find all combos summing to target. $[2, 3, 6, 7]$, $t=7 \rightarrow [[2, 2, 3], [7]]$.

Recall: Include: $arr[idx] \leq target \rightarrow push, recurse(idx, target - arr[idx]), pop$. Exclude: $recurse(idx+1, target)$. Recurse with idx (not $idx+1$) = reuse.

Trap: idx (same) in include branch is the ONLY change from P08. Without $arr[idx] \leq target$ guard $\rightarrow$ infinite recursion on positive elements.

P12 — Combination Sum II $O(2^N)/O(N)$

Problem: Array may have duplicates, each element used once; unique combos summing to target.

$[10, 1, 2, 7, 6, 1, 5]$, $t=8 \rightarrow [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]$.

Recall: Sort. Loop $i = idx \rightarrow n$: skip $i > idx \ \&\& \ arr[i] == arr[i-1]$; break $arr[i] > target$; push; $recurse(i+1, target - arr[i])$; pop.

Trap: $i > idx$ NOT $i > 0$. break (not continue) when $arr[i] > target$ — sorted means all after are also too large.

P13 — Subset Sums I $O(2^N \cdot N)/O(N)$

Problem: Find sum of every possible subset. Return sorted. $[5, 2, 1] \rightarrow [0, 1, 2, 3, 5, 6, 7, 8]$.

Recall: Include/exclude with `int currentSum` as parameter. Base $idx == n \rightarrow push currentSum$. Sort result. No pop_back — int passed by value.

Trap: Int param by value = no backtrack needed. Pass by reference would require += then -=. Don't forget to sort at the end.

P14 — Subsets II $O(2^N \cdot N)/O(N)$

Problem: Array with duplicates $\rightarrow$ all unique subsets. $[1, 2, 2] \rightarrow 6$ subsets.

Recall: Sort. `result.push_back(current)` on entry (every prefix = valid subset). Loop with same dup skip as P12. Recurse $i+1$.

Trap: "Push on entry" not "at base". Only difference from basic subsets (LC 78) is the one `continue` dup-skip line. $i > idx$ same rule.

P15 — Combination Sum III $O(C(9,k) \cdot k) / O(k)$

Problem: k distinct digits from 1–9 summing to n, each once. k=3, n=7 → [[1,2,4]].

Recall: Loop `i=start→9`. Success: `size==k && target==0`. Fail: `size>k || target<0`. If `i≤target`: `push/recurse(i+1)/pop`. Else: `break`.

Trap: Loop to 9, not n-1. Two separate base cases (success AND failure). `size>k` fail prune avoids over-picking.

P16 — Letter Combinations of Phone Number $O(4^N \cdot N) / O(N)$

Problem: Given digit string, return all letter combinations from phone keypad. "23" → 9 strings.

Recall: `compos[digit-'0']` maps digit to letters. Loop letters, `recurse(idx+1, s+letter)`. Base: `idx==size` → `push`. Guard: `digits.empty()` → `return {}`.

Trap: Empty guard is critical — without it, empty input pushes "" instead of returning {}. Passing s by value avoids explicit `pop_back`.

P17 — Palindrome Partitioning $O(2^N \cdot N) / O(N)$

Problem: Partition string so every substring is a palindrome. Return all partitionings. "aab" → [{"a","a","b"}], [{"aa","b"}].

Recall: Loop `end=start→n`. If `isPalindrome(start,end)`: `push substr(start,end-start+1)`, `recurse(end+1)`, `pop`. Base: `start==n` → `push curr`.

Trap: `substr(start, end-start+1)` — length is `end-start+1`, NOT `end`. Optimize: precompute `dp[i][j]` in $O(N^2)$ for $O(1)$ checks.

P18 — Word Search $O(M \cdot N \cdot 4^L) / O(L)$

Problem: Find if a word exists in a grid using adjacent cells (4-dir), each cell once per path.

Recall: Try all start cells. `dfs(i,j,idx)`: base `idx==len` → `true`. Guard OOB or mismatch. `board[i][j]=='#'`. Explore 4 dirs. Restore. Return found.

Trap: Mark BEFORE the 4-dir calls — cell can revisit itself without it. `||` short-circuits — stops once any direction returns true.

P19 — N-Queens $O(N!) / O(N)$

Problem: Place N non-attacking queens on N×N board. Return all solutions. n=4 → 2 solutions.

Recall: Column-by-column. Optimal: `leftRow[r]`, `lowerDiag[r+c]`, `upperDiag[n-1+c-r]`. Set all 3, recurse, unset all 3.

Diagonal keys: `row+col` constant along ↗; `col-row` constant along ↖ (offset by n-1). Array size: `2n-1`.

Trap: Array size `2n-1` not `n`. Forgetting to reset any of 3 arrays = wrong subsequent placements.

P20 — Rat in a Maze $O(4^N) / O(N^2)$

Problem: All paths from (0,0) to (n-1,n-1) in a binary grid, returning path as direction strings in lex order.

Recall: `isSafe`: in-bounds + maze=1 + not visited. `visited[x][y]=1`. Try D/L/R/U. `visited[x][y]=0`. Base: reach (n-1,n-1).

Trap: D/L/R/U order (alphabetical) naturally gives lex output. Guard `maze[0][0]==1`. Path passed by value — no explicit backtrack on string.

P21 — Word Break $O(N^3) / O(N)$

Problem: Can string be segmented into dictionary words? "leetcode", `dict=[leet,code]` → true.

Recall: `unordered_set` for $O(1)$ lookup. Loop `end=start+1→n`: if `dict.count(substr) && solve(end)` → cache 1. Base `start==n` → 1. `memo[start]`.

Trap: Same loop skeleton as P17. Without memoization: exponential. `substr(start, end-start)` length argument is `end-start`.

P22 — M Coloring Problem $O(M^N \cdot N)/O(N)$

 **Problem:** Can graph be colored with M colors, no two adjacent nodes same color? Return true/false.

 **Recall:** Node by node. Try colors $1 \rightarrow M$. `isSafe` : no adjacent node has same color. Assign, `recurse(node+1)` , backtrack `color=0` . Base `node==N` $\rightarrow$ true .

 **Trap:** Colors from 1 (not 0) — 0 = unassigned. Must reset `color[node]=0` after failed recursion branch.

P23 — Sudoku Solver $O(9^{\text{empty}})/O(1)$

 **Problem:** Fill a 9×9 Sudoku board in-place. '.' = empty. Every row, col, 3×3 box has digits 1–9 exactly once.

 **Recall:** Find first '.' . Try '1'→'9' . `isValid` : row + col + box ($3*(r/3)$, $3*(c/3)$). Place, recurse $\rightarrow$ if false: `board[i][j]='.'` . No '.' left $\rightarrow$ return true.

 **Trap:** return false inside the `if('.')` block triggers parent backtrack. Box start: $3*(\text{row}/3)$ not `row/3` .

P24 — Expression Add Operators $O(N \cdot 4^N)/O(N)$

 **Problem:** Insert +/−/* between digits to hit target. "123", t=6 $\rightarrow$ ["1+2+3", "1*2*3"] .

 **Recall:** DFS on all splits from start . First number: no operator. Else: `+(last=cur)` , `-(last=-cur)` , `*(val=val-last+last*cur; last=last*cur)` . Base: `start==n` && `val==target` .

 **Trap:** Leading zero: `i>start` && `num[start]=='0'` $\rightarrow$ return (not continue) . Use long long . Handle `start==0` separately.

KEY COMPARISON TABLES

Combination Sum Family:

	SUM I (P11)	SUM II (P12)	SUM III (P15)
Pool	<code>candidates[]</code>	<code>candidates[]</code> (sort first)	digits 1–9
Reuse	✓ yes	✗ no	✗ no
Dups	✗ no	✓ yes	✗ no
Recurse with	<code>idx</code> (same)	<code>i+1</code>	<code>i+1</code>
Dup skip	—	<code>i>idx</code> && <code>arr[i]==arr[i-1]</code>	—
Extra constraint	none	none	<code>size == k</code>

Grid DFS:

	WORD SEARCH (P18)	RAT IN MAZE (P20)
Mark	in-place '#'	<code>visited[][]</code> array
Restore	<code>board[i][j]=temp</code>	<code>visited[x][y]=0</code>
Goal	match word chars	reach destination
Collect	bool return	path direction string

Constraint Satisfaction:

	N-QUEENS (P19)	M-COLORING (P22)	SUDOKU (P23)
Fill order	column by column	node by node	cell by cell (row-major)
Options	N rows	M colors ($1 \rightarrow M$)	9 digits ('1' $\rightarrow$ '9')
Safety check	row + 2 diagonals	no adj same color	row + col + 3×3 box
Backtrack value	'.'	0	'.'

TOP 10 TRAPS

TRAP	WHERE IT BITES	FIX
------	----------------	-----

<code>i > 0</code> for dup skip	P12, P14 — blocks deeper levels	<code>i > idx</code>
<code>idx</code> instead of <code>i+1</code> in recurse	P12 — accidentally allows reuse	<code>i+1</code> for single-use
<code> </code> instead of <code> </code>	P10 — no short-circuit	Use <code> </code> for existence checks
Leading zero <code>continue</code> not <code>return</code>	P24 — still tries longer numbers	<code>return</code> to prune whole branch
Missing <code>pop_back()</code>	Any backtracking — state leaks	Every push needs a pop
<code>sum<0</code> prune on negative arrays	P09/P10 — invalid pruning	Remove for non-positive arrays
<code>int</code> for large exponent	P02, P24 — overflow	Use <code>long</code> / <code>long long</code>
Mark visited AFTER recurse	P18/P20 — cell revisits itself	Mark BEFORE 4-dir calls
<code>sum==0</code> after <code>ind==n</code> check	P09 — loses early-exit	Check <code>sum==0</code> first
No empty-input guard	P16 — pushes <code>""</code> not <code>{}</code>	<code>if(digits.empty()) return {}</code>